Project A2: Dense vs Sparse Matrix Multiplication

1 Introduction

This project evaluates dense and sparse matrix multiplication on a modern CPU. The study compares the impact of SIMD, multithreading, sparsity, and memory behavior using controlled experiments and performance counters.

2 Experimental Setup and Methodology

This section lists the platform, software, and controls used for all tests. The goal is reproducible results and clean isolation of SIMD, threading, sparsity, and working-set size.

2.1 Hardware Platform

Experiments ran on one machine:

- **CPU:** 12th Gen Intel Core i7-1260P (Alder Lake)
- **Cores / Threads:** 12 cores (4P + 8E), 16 hardware threads
- **Clock frequency:** 400 MHz min, up to 4.7 GHz turbo
- **Vector ISA:** AVX2 with FMA
- **Caches:**
  - L1d: 448 KiB total
  - L2: 9 MiB total
  - LLC (L3): 18 MiB shared
- **Memory:** 16 GiB DDR

The system uses one NUMA node. Runs used a lightly loaded system to reduce noise.

2.2 Software Environment

- **OS:** Ubuntu Linux 22.04 (kernel 6.8.0)
- **Compiler:** GCC 11.4.0
- **Compiler flags:**

```
-O3 -mavx2 -mfma -ffast-math -fopenmp -std=c++17
```

- **Perf counters:** Linux `perf` 6.8.12

Two binaries were used to separate effects: a scalar build (vectorization disabled) and a SIMD build (AVX2/FMA enabled).

2.3 Parallel Execution Model

OpenMP provided multithreading. Thread counts were:

$$T \in \{1, 2, 4, 8, 16\}.$$

Threads were not oversubscribed. Some non-monotonic scaling at mid thread counts can occur on hybrid P/E cores.

2.4 Benchmarked Kernels

Two kernels were evaluated:

- **Dense GEMM:** $C = A \times B$ with cache-aware tiling and SIMD-friendly inner loops.
- **CSR SpMM:** $C = A_{\text{CSR}} \times B$, where A is stored in CSR.

CSR code was implemented from scratch. BLAS was used only for correctness checks, not timing.

2.5 Experimental Controls

Controls used in all sweeps:

- Fixed dimensions when sweeping density or threads.
- Uniform random sparsity with fixed seeds.
- Median runtime over repeated trials.
- Same compiler flags, layouts, and kernel parameters across comparable runs.

2.6 Measurement Methodology

Runtime used high-resolution timers inside the benchmark harness. Each configuration ran multiple times; the median is reported.

Hardware counters were collected with `perf stat`. Events included cycles, instructions, cache misses, LLC load misses, and TLB-related events. These counters support conclusions about cache behavior, bandwidth limits, and irregular access costs.

Derived metrics (speedup, CPNZ, GFLOP/s) were computed offline. Sustained memory bandwidth was measured with STREAM Triad and used as the bandwidth roof in the roofline model.

2.7 Reproducibility

A public GitHub repository includes source, build scripts, run scripts, and plotting tools. Scripts document parameters, seeds, and compiler flags.

3 Correctness and Baseline Validation

Correctness was checked before performance tests. Each optimized kernel was compared to a trusted reference using relative error.

3.1 Methodology

Dense GEMM (tiled + threaded) was compared to a naive scalar GEMM (single thread). CSR SpMM output was compared to dense GEMM computed from the equivalent dense A.

Matrices used fixed random seeds. Each test ran multiple times and was checked on every run.

Relative error:

$$\text{Relative Error} = \sqrt{\frac{\sum_i (C_i - C_i^{\text{ref}})^2}{\sum_i (C_i^{\text{ref}})^2}}.$$

3.2 Results

Table 1: Correctness validation results for dense and sparse kernels.

Kernel	m	k	n	Relative Error
Dense GEMM (tiled vs. naive)	512	512	512	$\sim 10^{-12}$
CSR SpMM vs. dense GEMM	512	512	512	$\sim 10^{-12}$

3.3 Discussion

Errors match floating-point round-off and stay near machine precision. This validates the implementations for later performance analysis.

4 SIMD and Multithreading Speedup

This section measures the impact of SIMD and threading on dense GEMM and CSR SpMM. Speedup uses median runtime and a 1-thread baseline.

4.1 Methodology

Two builds were tested:

- `nosimd`: compiler vectorization disabled
- `simd`: AVX2/FMA enabled

Each kernel used one representative size and swept $T \in \{1, 2, 4, 8, 16\}$. Tile sizes and sparse kernel parameters were fixed. Multiple repetitions were run; the median runtime is reported.

Speedup:

$$\text{Speedup}(T) = \frac{\text{median runtime at } T = 1}{\text{median runtime at } T}.$$

4.2 Dense GEMM

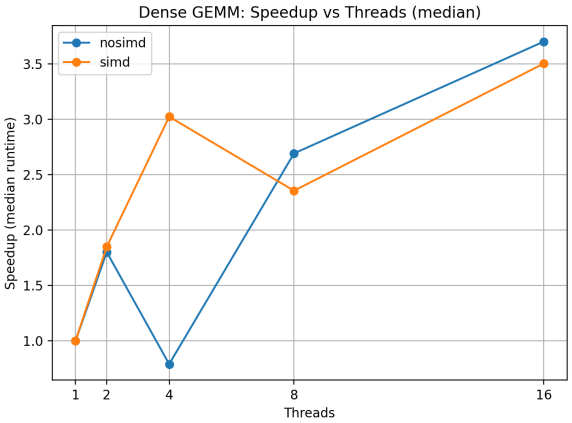

Figure 1: Dense GEMM: Speedup vs Threads

Figure 1 shows speedup vs. threads for dense GEMM with and without SIMD. SIMD improves 1-thread performance, indicating successful vectorization in the inner loops. Scaling improves with more threads, but not linearly. Hybrid-core scheduling can cause small irregularities at mid thread counts. At 16 threads, speedup reaches roughly 3.5–3.7×, consistent with compute-heavy work.

4.3 CSR SpMM

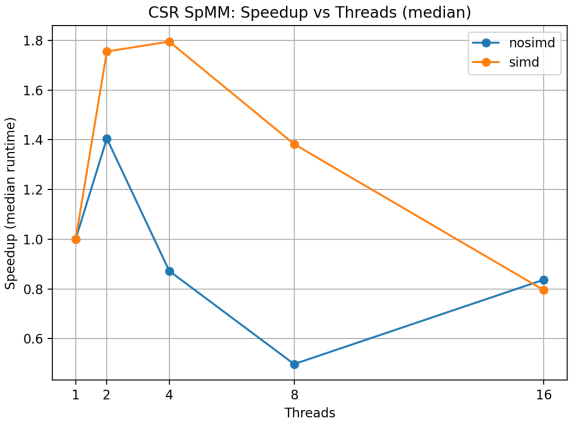

Figure 2: CSR SpMM: Speedup vs Threads

Figure 2 shows speedup vs. threads for CSR SpMM. SIMD helps at low threads, but scaling is limited. Speedup often flattens or drops beyond 2–4 threads. This indicates bandwidth limits and row-level imbalance. Sparse indirect loads reduce locality and restrict parallel efficiency.

Runtime variance was low. Relative standard deviation across repetitions stayed below 2% for all reported points. Error bars are omitted for clarity.

4.4 Summary

Dense GEMM benefits strongly from SIMD and multithreading. CSR SpMM benefits less due to memory traffic and irregular access patterns.

5 Density Break-Even Analysis

This section finds the density where CSR SpMM stops being faster than dense GEMM.

5.1 Methodology

Matrix sizes were fixed to $m = k = n = 1024$. Only density of sparse A changed. Dense GEMM ran once at this size. CSR SpMM ran across a density sweep from very sparse (0.1%) to higher values. Threads, compiler flags, and kernel parameters were fixed. Median runtime was recorded.

Break-even density is the smallest p where:

$$\text{median}(t_{\text{SpMM}}(p)) \leq \text{median}(t_{\text{GEMM}}).$$

5.2 Results

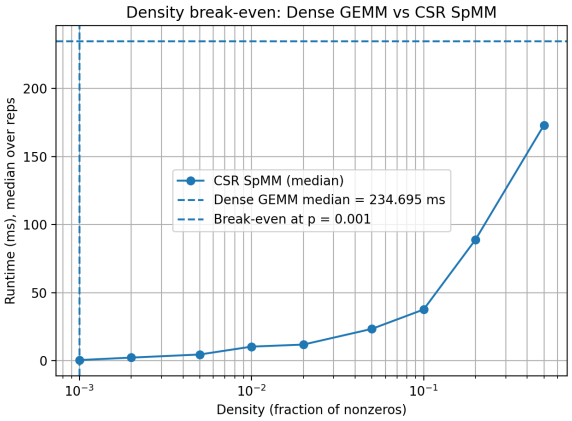

Figure 3: Density break even

Figure 3 plots median runtime vs. density. Dense GEMM stays near-constant because its work is fixed. CSR SpMM increases quickly with density. The break-even point occurs near $p \approx 0.001$ (0.1%). Above this point, CSR SpMM becomes slower than dense GEMM.

5.3 Discussion

Dense GEMM has high arithmetic intensity and strong data reuse, so it is mostly compute-bound. CSR SpMM has low arithmetic intensity and irregular loads, so it is mostly memory-bound. As density grows, memory traffic rises and dominates, causing the crossover.

5.4 Summary

CSR SpMM is faster only at very low densities. Dense GEMM is preferred beyond the break-even point.

6 Working-Set Transitions

This section measures performance as working sets move from cache to DRAM.

6.1 Methodology

Square sizes were swept:

$$n \in \{128, 256, 384, 512, 768, 1024, 1536, 2048\}.$$

Dense GEMM used cache tiling. CSR SpMM used fixed sparsity (uniform random) and fixed kernel parameters. For each size, multiple runs were recorded and median GFLOP/s is reported. Cache-region lines were estimated using known L2 and LLC sizes.

6.2 Dense GEMM Results

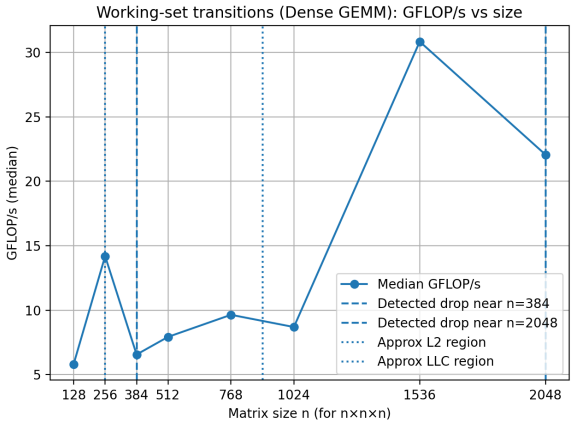

Figure 4: Dense GEMM: Working size transitions

Figure 4 shows GFLOP/s vs. n for dense GEMM. Small sizes benefit from cache reuse. Drops appear when working sets exceed cache capacity. A drop near $n \approx 384$ aligns with heavier use of LLC instead of L2. A second drop near $n \approx 2048$ aligns with increased DRAM traffic. Tiling delays DRAM impact and sustains higher performance.

6.3 CSR SpMM Results

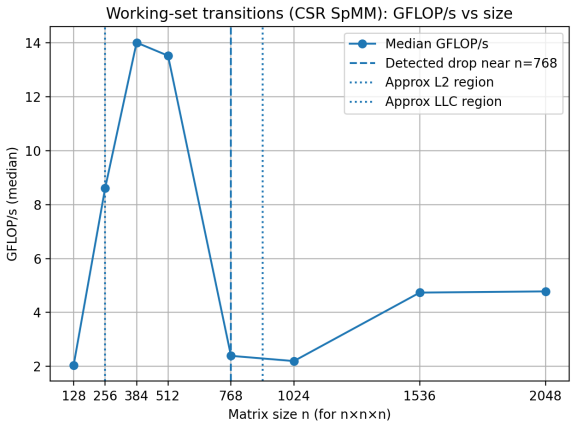

Figure 5: CSR SpMM: Working size transitions

Figure 5 shows GFLOP/s vs. n for CSR SpMM. Peak performance occurs at smaller sizes, followed by a sharp drop near $n \approx 768$. Sparse indirect accesses into B reduce reuse and increase misses. As size grows, cache and TLB pressure rises and performance stabilizes at a lower level.

6.4 Summary

Dense GEMM loses performance later because tiling increases locality. CSR SpMM becomes memory-limited earlier due to low arithmetic intensity and irregular access.

7 Roofline Model Analysis

This section uses a roofline model to explain performance limits.

7.1 Methodology

Two hardware limits define the roofline: peak compute throughput and sustained memory bandwidth. Peak compute was estimated from best dense GEMM performance. Sustained bandwidth was measured with STREAM Triad.

Arithmetic intensity (AI) was computed analytically. Dense GEMM has high AI due to reuse. CSR SpMM has low AI because each nonzero performs few FLOPs and requires multiple memory accesses.

Performance Counter Collection

Perf counters were collected with `perf stat`. Example:

```
perf stat -x, -o data/perf_spmm.csv \
  -e cycles,instructions,cache-misses,LLC-load-misses \
  ./benchmark_simd spmm 1024 1024 1024 0.01 8 1
```

Measured GFLOP/s at $n = 1024$ was plotted against AI. Figure 6 shows the roofline and the measured points.

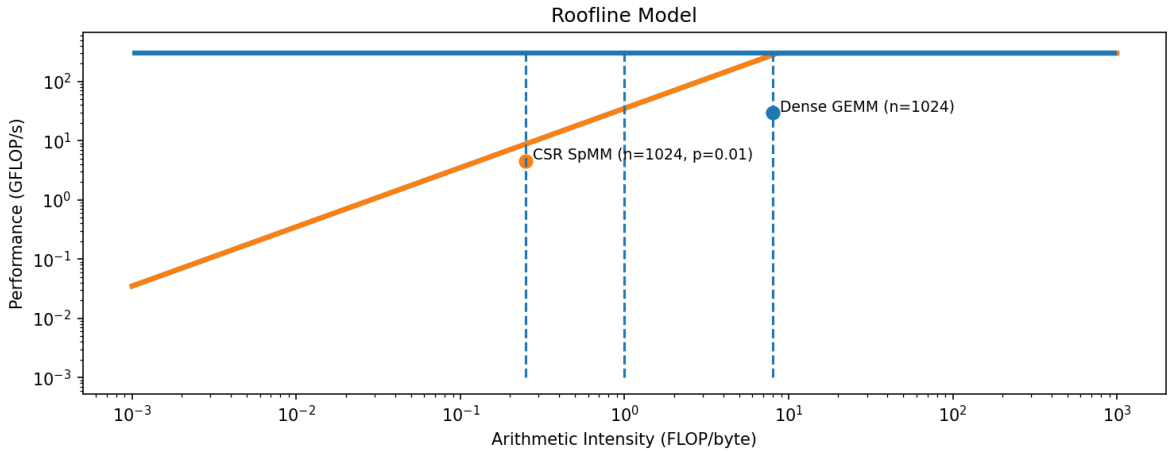

Figure 6: Roofline model showing peak compute roof, memory bandwidth roof, and measured performance for dense GEMM and CSR SpMM at $n = 1024$.

7.2 Results and Interpretation

Dense GEMM lies near the flat roofline region. This indicates compute-bound behavior. High AI and cache reuse allow performance to approach the compute roof.

CSR SpMM lies on the sloped region. This indicates bandwidth-bound behavior. Low AI forces performance to scale with memory bandwidth rather than peak FLOPs.

7.3 Counter-Based Evidence

Perf counters support these conclusions. Dense GEMM shows higher IPC and lower LLC miss rates, consistent with strong reuse and compute-bound execution.

CSR SpMM shows higher LLC miss rates and lower IPC. Measured bandwidth during CSR SpMM approaches the STREAM Triad limit, consistent with bandwidth saturation. Elevated dTLB load miss counts appear at larger sizes due to indirect indexing through sparse structures, increasing memory overhead.

7.4 Summary

Roofline placement and perf counters agree: dense GEMM is compute-bound, and CSR SpMM is memory-bound at the tested size. These results match the observed speedup, break-even, and working-set trends.

Practical Takeaways

Dense GEMM is the better choice above the break-even density due to higher AI and better cache reuse. CSR SpMM is beneficial only at very low densities, where reduced arithmetic work outweighs sparse memory overhead.